

Get results

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/coffees`
- Method:** `GET`
- Status:** `200 OK` (8 ms, 242 B)
- Body:** A JSON array containing two coffee objects.

```
1  [
2    {
3      "id": 1,
4      "name": "Espresso",
5      "price": 45.0
6    },
7    {
8      "id": 2,
9      "name": "Latte",
10     "price": 55.0
11   }
12 ]
```

GET results by id

The screenshot shows a REST client interface with the following components:

- Overview** tab selected, showing the request `GET http://localhost:8080/c`.
- Request URL: `http://localhost:8080/coffees/1`
- Method: `GET`
- Buttons: `Save`, `Share`, `Send`
- Tabs: `Docs`, `Params`, `Authorization`, `Headers` (6), `Body` (selected), `Scripts`, `Tests`, `Settings`
- Body format: `raw` (selected), `JSON`
- Buttons: `Schema`, `Beautify`
- Response status: `200 OK`, `9 ms`, `203 B`
- Response body (JSON):

```
{  "id": 1,  "name": "Espresso",  "price": 45.0}
```

Post Menu

GET http://localhost:8080/caffeines

Body

```
1 {
2   "id": 3,
3   "name": "yoru",
4   "price": 70.0
}
```

Body

```
1 [
2   {
3     "id": 1,
4     "name": "Espresso",
5     "price": 45.0
6   },
7   {
8     "id": 2,
9     "name": "Latte",
10    "price": 55.0
11  },
12  {
13    "id": 3,
14    "name": "yoru",
15    "price": 70.0
16  },
17  {
18    "id": 4,
19    "name": "yoru",
20    "price": 70.0
21  }
22 ]
```

200 OK · 4 ms · 314 B

Push

PUT http://localhost:8080/caffeines/2

Body

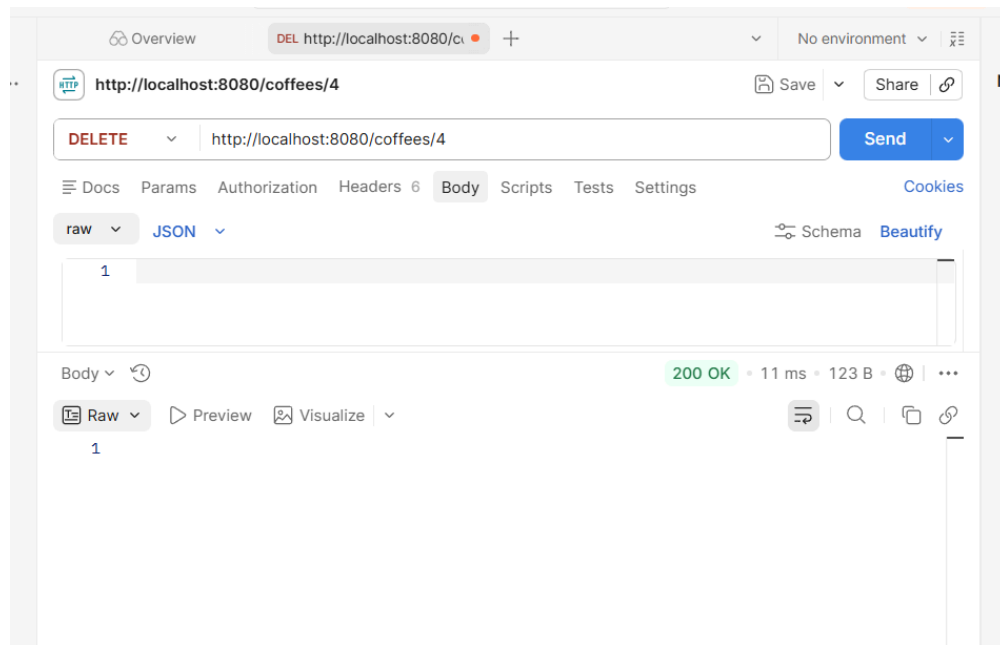
```
1 { "name": "Latte", "price": 50.0 }
```

Body

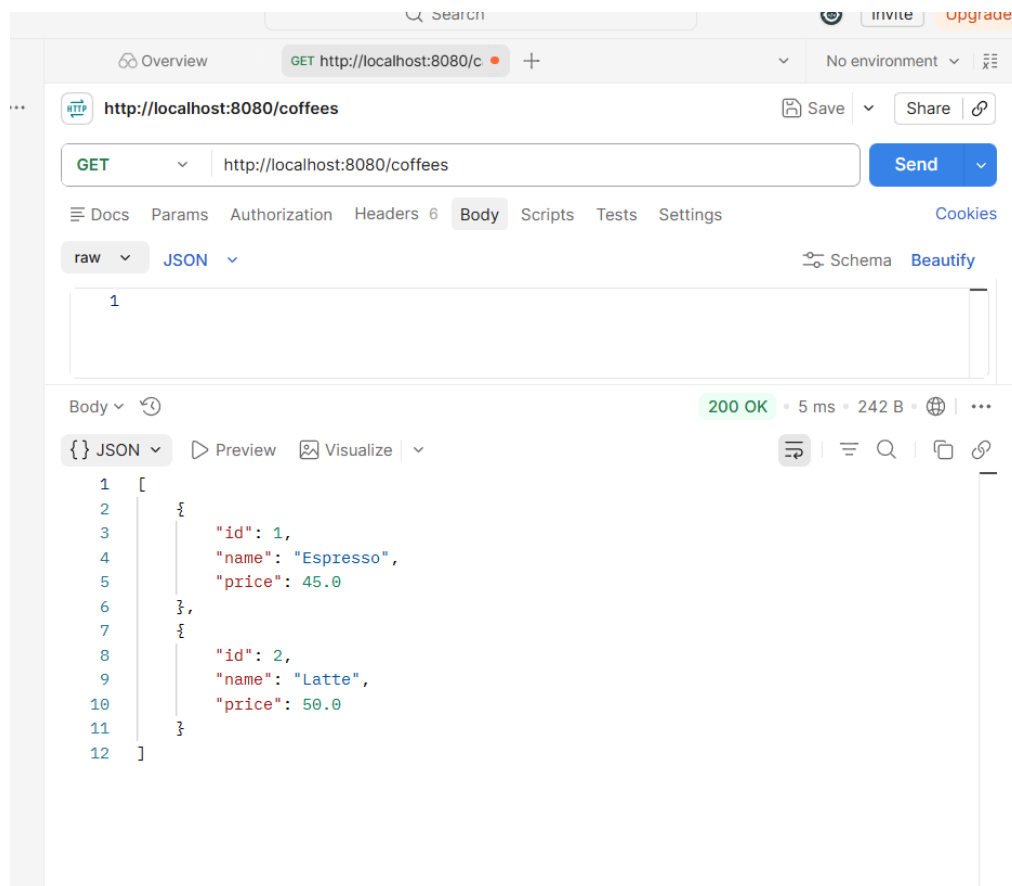
```
1 {
2   "id": 2,
3   "name": "Latte",
4   "price": 50.0
5 }
```

200 OK · 9 ms · 200 B

Delete



Before delete



Bonus score

คืน 404 Not Found เมื่อหา id ไม่เจอ

localhost:8080/coffees/3



ไม่พบหน้า localhost นี้

ไม่พบหน้าเว็บสำหรับที่อยู่เว็บ: <http://localhost:8080/coffees/3>

HTTP ERROR 404

โหลดใหม่

Discussion

1. HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

GET – ใช้สำหรับ ดึงข้อมูล

- ตัวอย่าง: GET /coffees/{id}
- โค้ด: `coffeeService.findById(id)`
- ความหมาย: รับ id ของกาแฟ แล้วคืนข้อมูลกลับมา ถ้าไม่พบจะส่ง 404 Not Found

POST – ใช้สำหรับ เพิ่มข้อมูลใหม่

- ตัวอย่าง: POST /coffees
- โค้ด: `coffeeService.create(coffee)`
- ความหมาย: รับข้อมูลกาแฟจาก Request Body แล้วสร้างข้อมูลใหม่ในระบบ จากนั้นส่งกลับ 201 Created

PUT – ใช้สำหรับ แก้ไขข้อมูลเดิม

- ตัวอย่าง: PUT /coffees/{id}
- โค้ด: `coffeeService.update(id, coffee)`
- ความหมาย: รับ id และข้อมูลใหม่ แล้วอัปเดตข้อมูลกาแฟ ถ้าไม่พบข้อมูลจะส่ง 404 Not Found

DELETE – ใช้สำหรับ ลบข้อมูล

- ตัวอย่าง: DELETE /coffees/{id}
- โค้ด: `coffeeService.delete(id)`

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

เพื่อแบ่งหน้าที่ให้ชัดเจน โดย Controller รับ-ส่งข้อมูล ส่วน Service จัดการ Business Logic ทำให้โค้ดเป็นระเบียบ แก้ไข ทดสอบ และขยายระบบได้ง่ายเมื่อโปรแกรมมีขนาดใหญ่ขึ้น

3. ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้มีไม่หายควรทำอย่างไร (ตอบเป็นแนวคิดพอ)

ข้อมูลใน List จะหายเมื่อโปรแกรมปิดหรือเซิร์ฟเวอร์รีสตาร์ท เพราะเก็บอยู่ใน Memory (RAM) ถ้าต้องการให้ข้อมูลคงอยู่ควรเก็บใน Database หรือไฟล์ถาวรแทน

4. @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

@RestController → ประกาศว่าเป็น REST API

@GetMapping → ดึงข้อมูล (GET)

@PostMapping → เพิ่มข้อมูล (POST)

@PathVariable → รับค่าจาก URL

@RequestBody → รับข้อมูล JSON จาก Body แล้วแปลงเป็น Object